# BÁO CÁO CÔNG NGHỆ:DOCKER

PHẠM TÂM CHÍNH

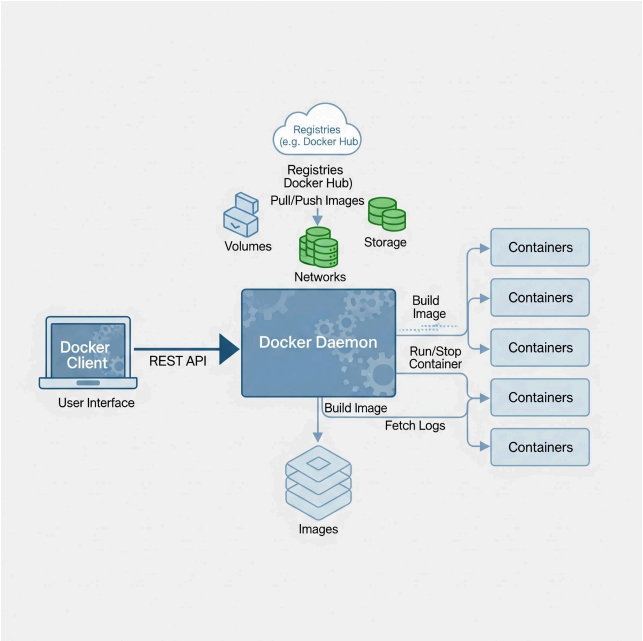
## I. TỔNG QUAN VỀ DOCKER

### 1. Docker là gì?

Docker là một nền tảng mở cho phép phát triển, vận chuyển và chạy các ứng dụng. Docker cho phép tách biệt ứng dụng khỏi cơ sở hạ tầng (infrastructure), giúp việc chuyển giao phần mềm nhanh chóng hơn.

Vấn đề lớn nhất mà Docker giải quyết là câu chuyện kinh điển trong ngành phần mềm: ***"Nó chạy tốt trên máy của tôi, nhưng không chạy trên máy server" (It works on my machine)***. Điều này xảy ra do sự khác biệt về môi trường Hệ điều hành (OS), phiên bản thư viện và các phụ thuộc (dependencies) giữa các máy khác nhau.

Docker giải quyết triệt để vấn đề này bằng cách đóng gói ứng dụng và tất cả các phụ thuộc của nó vào một "Container" – một đơn vị phần mềm được chuẩn hóa, đảm bảo nó chạy giống hệt nhau ở mọi nơi.



# II. DOCKERFILE VÀ DOCKER IMAGES

## 1. DOCKERFILE

Dockerfile là một tệp văn bản đơn giản (không có đuôi file) chứa một loạt các hướng dẫn (instructions) mà Docker Engine sẽ đọc tuần tự để tự động xây dựng nên một Docker Image.

```
# 1. FROM: Luôn bắt đầu bằng việc chọn Base Image (Hệ điều hành nền)
FROM python:3.10-slim

# 2. Environment Variables:
# PYTHONDONTWRITEBYTECODE: Ngăn Python tạo file .pyc (không cần thiết trong container)
# PYTHONUNBUFFERED: Quan trọng cho AI/Logs. Đảm bảo logs được in ra ngay lập tức thay vì bị buffer (giữ lại).
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

# 3. WORKDIR: Thiết lập thư mục làm việc
WORKDIR /app

# 4. Install System Dependencies (Nếu cần) và các thư viện khác:
RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc \
    && rm -rf /var/lib/apt/lists/*

# 5. COPY & INSTALL (Cơ chế Layer Caching):
# Copy file requirements trước để tận dụng cache nếu không thay đổi thư viện
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# 6. COPY Source Code:
COPY . .

# 7. CMD: Chạy Agent
CMD ["python", "agent_main.py"]
```

*Hình 1: Ví dụ Dockerfile cơ bản*

Các chỉ thị cốt lõi:

- **FROM**: Chỉ định xuất phát điểm
- **WORKDIR**: Giống như lệnh cd trong Linux, nhưng an toàn hơn. Nó đảm bảo mọi thao tác file đều nằm gọn trong thư mục dự án, không làm rác hệ thống file gốc của container.
- **COPY vs ADD**: Cả hai đều chép file từ máy Host vào Container. Tuy nhiên, nên dùng COPY vì nó rõ ràng và đơn giản. ADD có thêm tính năng giải nén file tar tự động hoặc tải file từ URL, dễ gây nhầm lẫn.
- **RUN**: Dùng để chạy các lệnh cài đặt trong quá trình build image (Ví dụ: apt-get install, pip install). Kết quả của lệnh này được lưu lại thành một Layer mới.
- **CMD vs ENTRYPOINT**:
    - **CMD**: Cung cấp lệnh mặc định, nhưng người dùng có thể ghi đè khi chạy docker run.
    - **ENTRYPOINT**: Cấu hình container để chạy như một file thực thi cố định.

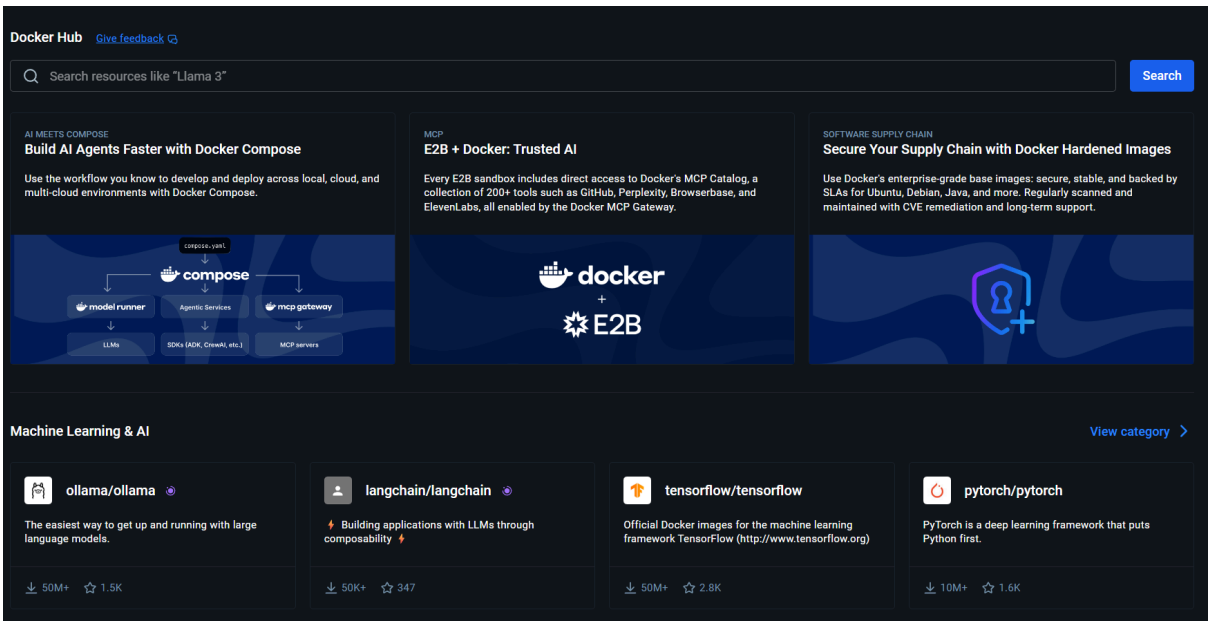
## 2. DOCKER IMAGES

Docker Image là một mẫu (template) chỉ đọc (read-only) được xây dựng từ Dockerfile. Nó chứa mã nguồn, thời gian chạy (runtime), thư viện, biến môi trường và các tệp cấu hình cần thiết để chạy ứng dụng.

**Kiến trúc phân lớp (Layered Architecture):** Một đặc điểm kỹ thuật quan trọng là Docker Image được xây dựng từ nhiều lớp (layers) xếp chồng lên nhau. Về cơ bản, mỗi chỉ thị trong Dockerfile (như RUN, COPY) sẽ tạo ra một lớp mới.

- **Lợi ích của Layer:** Khi chúng ta thay đổi một dòng trong Dockerfile và build lại, Docker sẽ sử dụng cơ chế Caching (bộ nhớ đệm). Nó chỉ xây dựng lại các lớp từ điểm thay đổi đó trở đi, giúp tiết kiệm đáng kể thời gian build và dung lượng lưu trữ.

# III. DOCKER HUB




*Hình 2: Giao diện Docker Hub*

Docker Hub là một dịch vụ đăng ký (Registry) dựa trên đám mây lớn nhất thế giới. Nó hoạt động tương tự như GitHub nhưng dành cho Docker Images. Đây là nơi cộng đồng nhà phát triển, các tổ chức phần mềm mã nguồn mở và các công ty độc lập (ISV) xây dựng và chia sẻ các image container của họ.

Lợi ích của Docker Hub:

- **Kho chứa ảnh chính thức (Official Images):** Docker Hub cung cấp các image chuẩn, được kiểm duyệt về bảo mật từ các nhà cung cấp chính thống (ví dụ: nginx, mongo, node, python, ubuntu). Luôn nên ưu tiên sử dụng các image này làm Base Image (FROM) để đảm bảo an toàn và hiệu năng.
- **Lưu trữ riêng tư (Private Repositories):** Cho phép các công ty lưu trữ các image chứa mã nguồn nội bộ (proprietary code) mà không công khai ra cộng đồng bên ngoài.

- **Tự động hóa (Automation Build & Webhooks):** Tích hợp chặt chẽ với các hệ thống CI/CD (như Jenkins, GitHub Actions) để tự động build và đẩy image mới lên Hub ngay khi có thay đổi trong mã nguồn (commit).

# IV. CÁC CÂU LỆNH DOCKER(CLI)

## 1. QUẢN LÝ LIFECYCLE CONTAINER

**docker run <image_name>**: Lệnh quan trọng nhất. Nó đảm nhiệm việc tạo mới một container từ image và khởi chạy nó.

- Thường dùng kèm các flags (cờ):
  - **-d** (detach): Chạy container ở chế độ nền (background/ngầm), trả lại terminal cho người dùng.
  - **-p host_port:container_port** (publish): Ánh xạ cổng từ máy host vào cổng của container để có thể truy cập ứng dụng từ bên ngoài.
  - **--name <tên>**: Đặt một tên dễ nhớ cho container thay vì dùng ID ngẫu nhiên.

**docker ps**: Liệt kê các container đang ở trạng thái chạy (Up). (Thêm flag -a để xem tất cả, bao gồm cả các container đã dừng/thoát).

**docker stop <container_id/name>**: Yêu cầu một container đang chạy dừng lại một cách an toàn (gửi tín hiệu SIGTERM để ứng dụng lưu trạng thái và đóng kết nối).

**docker start <container_id/name>**: Khởi chạy lại một container đã bị dừng trước đó.

**docker rm <container_id/name>**: Xóa bỏ hoàn toàn một container đã dừng. (Lưu ý: Dữ liệu bên trong container sẽ mất nếu không dùng Volume. Thêm -f để buộc xóa container đang chạy).

### 2. QUẢN LÝ IMAGES

**docker build -t <tag_name> .**: Lệnh để Docker đọc Dockerfile trong thư mục hiện tại (.), thực hiện các chỉ dẫn để xây dựng image và gắn nhãn (tag) cho nó bằng -t.

**docker pull <image_name>**: Chủ động tải một image từ Docker Hub (hoặc registry khác) về máy local mà chưa chạy nó.

**docker images**: Liệt kê danh sách tất cả các image đang có sẵn trên máy local.

**docker rmi <image_id/name>**: Xóa một image khỏi máy local để giải phóng dung lượng ổ cứng.

## V. DOCKER VOLUMES

Theo thiết kế mặc định, dữ liệu bên trong container là "phù du" (ephemeral). Điều này có nghĩa là khi một container bị xóa (docker rm), mọi dữ liệu được tạo ra hoặc thay đổi bên trong lớp writable của nó cũng sẽ biến mất vĩnh viễn. Điều này là không thể chấp nhận được đối với các ứng dụng như Cơ sở dữ liệu (Database) hoặc khi cần lưu trữ file logs quan trọng.

**Giải pháp: Docker Volumes**. Đây là cơ chế được khuyến nghị để duy trì dữ liệu được tạo ra và sử dụng bởi Docker containers, tách biệt dữ liệu khỏi vòng đời của container.

Cách dùng (CLI): **docker run -v my_db_data:/var/lib/mysql mysql** (Tạo một named volume tên là my_db_data và gắn nó vào thư mục dữ liệu của MySQL trong container).

# VI. DOCKER COMPOSE

## 1. Vấn đề thực tế

Trong môi trường thực tế, các ứng dụng hiện đại(microservices) hiểm khi chạy cô lập trong một container duy nhất.Một hệ thống tiêu chuẩn thường bao gồm nhiều thành phần hoạt động cùng nhau, ví dụ: Backend API + Frontend + Cơ sở dữ liệu (PostgreSQL/MySQL) + Caching (Redis) + Reverse Proxy (Nginx).

Nếu chỉ sử dụng Docker CLI thuần túy, để khởi chạy hệ thống này, ta phải thực hiện một chuỗi các thao tác phức tạp và dễ sai sót:

1. Tạo một Docker Network thủ công để các container nhìn thấy nhau.
2. Chạy container Database, nhớ đặt tên và thiết lập biến môi trường cho mật khẩu.
3. Chạy container Backend, sử dụng flag **--network** để kết nối vào mạng trên, dùng flag **-e** để truyền biến môi trường (kết nối DB).
4. Lặp lại quy trình cho Frontend và các dịch vụ khác.

Quy trình này rất khó quản lý, khó nhớ và cực kỳ bất tiện khi muốn chia sẻ môi trường phát triển cho thành viên mới trong team.

**Docker Compose** là công cụ ra đời để giải quyết vấn đề này. Nó cho phép định nghĩa và vận hành các ứng dụng Docker đa container. Nó giúp mô tả toàn bộ kiến trúc hệ thống (Infrastructure as Code) trong một tệp cấu hình YAML duy nhất.

## 2. Phân tích sâu

Tệp cấu hình YAML là "trái tim" của Docker Compose. Dưới đây là phân tích cấu trúc một file mẫu thực tế cho một ứng dụng Web cơ bản gồm Backend và Database:

```
version: "3.8"
```

```yaml
services:
  # 1. AI Agent Service (Python App)
  ai-agent:
    build: .
    container_name: my_ai_agent
    volumes:
      - ./src:/app/src       # Hot-reload code khi dev
      - ./.env:/app/.env     # Map file biến môi trường
    environment:
      # Cấu hình chuỗi kết nối tới Database
      - DATABASE_URL=postgresql://myuser:mypassword@postgres_db:5432/ai_app_db
      - OPENAI_API_KEY=${OPENAI_API_KEY}
    depends_on:
      - postgres_db         # Đợi DB chạy trước khi start App
    networks:
      - ai_network

  # 2. Database Service (PostgreSQL)
  postgres_db:
    image: postgres:15-alpine #Pull image thi docker hub
    container_name: ai_postgres_db
    environment:
      # Các biến bắt buộc để khởi tạo DB lần đầu
      - POSTGRES_USER=myuser
      - POSTGRES_PASSWORD=mypassword
      - POSTGRES_DB=ai_app_db
    ports:
      - "5432:5432" # Mở cổng ra máy host để dùng tool quản lý (DBeaver, pgAdmin) kết
nối vào
    volumes:
      - pg_data:/var/lib/postgresql/data # Volume để dữ liệu không bị mất khi tắt container
    networks:
      - ai_network

  # (Tùy chọn) Web UI để quản lý DB nhanh (Adminer)
  adminer:
    image: adminer
    ports:
      - "8080:8080"
    networks:
      - ai_network
```

```
# Định nghĩa Volume bền vững
volumes:
  pg_data:

# Định nghĩa Network riêng
networks:
  ai_network:
    driver: bridge
```